From Hand-Tuning to Architecture Search: CNN Design, Transfer Learning and Zero-Cost NAS for Satellite Land-Use Classification

Emilio Yared
Group 7
Master 2 Artificial Intelligence and Data Engineering
AIDE500 — Programming for AI

Joe Missisian
Group 7
Master 2 Artificial Intelligence and Data Engineering
AIDE500 — Programming for AI



Fig. 1. Two EuroSAT examples per class. Several classes differ mostly by texture and color (AnnualCrop / PermanentCrop / HerbaceousVegetation / Pasture), and Highway and River are both thin linear structures.

***Abstract*—We study how far a convolutional network trained from scratch can go on EuroSAT, a 10-class satellite land-use dataset, and compare three ways of obtaining a good model. First, a baseline CNN is improved by hand, one factor at a time: architecture (depth, kernel size, padding, downsampling, BatchNorm), optimizer, learning rate and scheduler, and data augmentation including MixUp and CutMix; test accuracy rises from 90.15% to 97.04%. Second, an ImageNet-pretrained ResNet-18 is transferred with frozen, partially and fully fine-tuned backbones on 5% to 100% of the training data; full fine-tuning reaches 99.04% and still 95.95% with only 945 training images. Third, we implement the training-free NAS score of Mellor et al. (NASWOT) and first reproduce the paper on NAS-Bench-201: our Kendall correlations (0.60/0.64/0.60) and search results (e.g. 92.83±0.94% vs. 92.81±0.99% on CIFAR-10 for N=100) match the published numbers. Applied to a EuroSAT search space of 8,640 CNNs, zero-cost NAS finds an architecture that beats our hand-designed network (97.43% vs. 96.72% under the same recipe) with 12× less GPU time.**

***Index Terms*—Convolutional neural networks, transfer learning, data augmentation, neural architecture search, remote sensing**

I. Introduction

Satellite images are a natural testbed for small convolutional networks: the classes (forest, river, highway, residential area…) are defined by texture and color rather than by object shape, the images are tiny (64×64 for EuroSAT [1]), and the dataset is large enough (27,000 images) to train a model from scratch. This project follows the three tasks of the course assignment — a baseline CNN (Task 1), a systematic study of architecture, optimization and augmentation (Task 2), and transfer learning (Task 3) — and adds a fourth question: **can the manual design of Task 2 be automated cheaply?** For this we implement zero-cost neural architecture search (NAS) [2], verify our implementation by reproducing the published results on NAS-Bench-201 [3], and then apply it to EuroSAT.

All experiments were run as Jupyter notebooks on a Kaggle T4 GPU (NB0–NB7); every trained configuration is saved as a checkpoint. Every choice (best configuration, best epoch) is made on a validation set; the test set is used only to report final numbers.

II. Data and protocol

EuroSAT RGB [1] contains 27,000 Sentinel-2 tiles of 64×64 pixels in 10 classes (2,000–3,000 per class). We use a fixed **stratified** 70/15/15 split (18,900 / 4,050 / 4,050 images) saved once in `splits.json`, and normalize with the per-channel mean and standard deviation of the **training** split only, to avoid leaking test statistics.

Fig. 1 shows the ten classes. The dataset is balanced enough for accuracy to be meaningful, but some classes are visually close, which is where most errors occur.

Metrics. For every epoch we log the training and validation loss, the validation accuracy and the per-class validation accuracy. We report the peak validation accuracy, the **generalization gap** (train − validation accuracy at the best epoch), the number of parameters, the inference latency for one image

(batch of 1, GPU synchronized before and after timing), the test accuracy and the macro-F1. Unless stated otherwise, every configuration is trained once with seed 42; repeating a run with the same configuration showed a run-to-run variation of about ±0.3–1.5 points, which we keep in mind when comparing close results.

III. Task 1 — Baseline CNN

The baseline (NB1) is a LeNet-style network written like the course labs: three blocks `Conv3×3(pad 1) → ReLU → MaxPool2×2` with 32, 64 and 128 channels (64 → 32 → 16 → 8 pixels), a flatten (`x.view(-1, 128*8*8)`) and two fully connected layers (8192 → 256 → 10), 2.19M parameters. It is trained for 30 epochs with SGD (momentum 0.9, lr 0.01, batch 64) and cross-entropy.

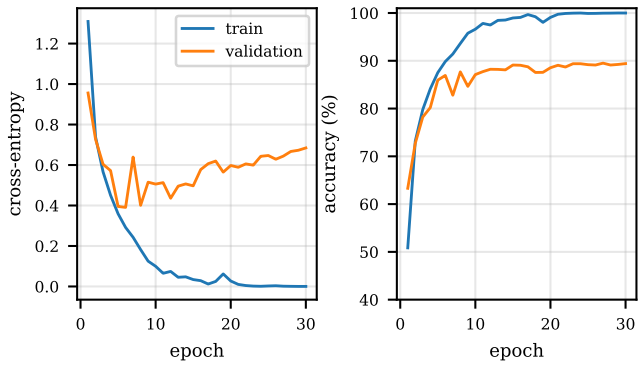


Fig. 2. NB1 baseline learning curves: the validation loss is minimal around epoch 6 and then rises while the training accuracy goes to 100%: the network overfits.

The baseline reaches 89.51% peak validation accuracy and **90.15% test accuracy**. Fig. 2 shows a textbook overfitting pattern: after epoch 6 the training loss keeps decreasing while the validation loss increases, and the final gap is 10.5 points. The confusion matrix shows that errors are concentrated on visually similar pairs: Highway→River (8.8% of the highways: both are thin linear structures), PermanentCrop→HerbaceousVegetation (8.3%) and Pasture/AnnualCrop. Overfitting and these confusions motivate the three axes of Task 2.

IV. Task 2 — Systematic study

A. Architecture (NB2)

Starting from the baseline, we change **one factor at a time** with a configurable `ConfigNet` (same training as NB1): depth (2–5 conv blocks, channels doubling from 32 up to 256), number of FC layers (1–3), kernel size (3/5/7, padding $k/2$), valid instead of same padding, stride-2 convolutions instead of max-pooling, and BatchNorm [4]. Finally, the best value of each group is combined. Results are in Fig. 3 and Table I.

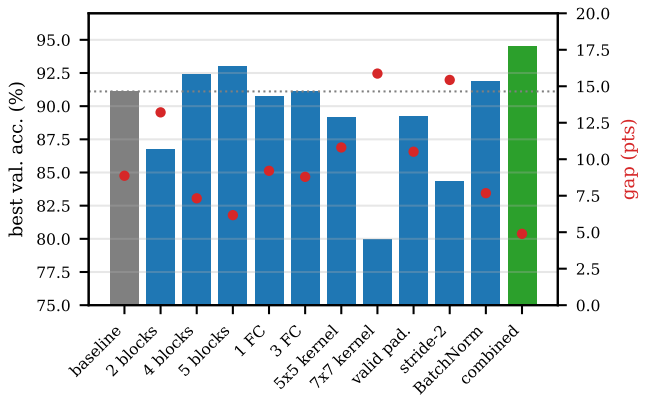


Fig. 3. NB2: peak validation accuracy (bars) and generalization gap (red dots) of each architecture variation.

Depth is the most important factor: each extra block doubles the receptive field (22 → 46 → 94 pixels for 3/4/5 blocks, i.e. the whole tile for 5 blocks) and divides the flattened features by 4, so **deeper networks are also smaller**: 5 blocks give +1.9 points with 1.24M parameters instead of 2.19M, and a smaller gap. With 2 blocks, the first FC layer alone has 4.2M weights and the model overfits the most. **Large kernels hurt** (7×7: 79.95%): with the same depth they add many parameters but no useful context, and training is slower. **Valid padding** loses the borders at every layer, and **stride-2 convolutions** (84.37%) discard 3/4 of the positions before the non-linearity, whereas max-pooling keeps the strongest response. **BatchNorm** gives +0.8 points and smoother training. The combination (5 blocks, 3×3, same padding, max-pooling, BatchNorm, 2 FC) reaches 94.52% validation and **95.36% test** accuracy, with a gap halved compared to the baseline (4.9 vs. 8.9 points in the same run).

B. Optimizers, learning rates and schedulers (NB3)

With the NB2 architecture fixed, we compare SGD with momentum (weight decay 5e-4), Adam [5] and AdamW [6] (weight decay 0.01) at three initial learning rates (25 epochs, Fig. 4). For all three optimizers **1e-3 is the best learning rate**. At 1e-2 the adaptive optimizers are unstable (Adam ends 1.5 points below its peak, AdamW 3 points), and at 1e-4 SGD is far too slow (23 epochs to reach 90% instead of 6–10). Adam with lr 1e-3 is best (94.15%) and converges fastest. With this pair, a scheduler adds +1.4–1.7 points: StepLR (÷10 every 10 epochs), cosine annealing [7] and OneCycle [8] reach 95.85/95.85/95.53%. Decreasing the learning rate at the end removes the epoch-to-epoch oscillations of the validation accuracy (Fig. 4, right). The selected recipe (Adam 1e-3 + StepLR) gives **96.22% test accuracy**.

TABLE I

TASK 2 COMPARISON TABLE: EVERY CONFIGURATION OF NB2 (ARCHITECTURE), THE BEST NB3 RUNS (OPTIMIZATION) AND THE NB4 RUNS (AUGMENTATION). PEAK VAL. = BEST VALIDATION ACCURACY (%); GAP = TRAIN − VALIDATION ACCURACY AT THE BEST EPOCH (NB4: CLEAN TRAIN ACCURACY).

NB	Configuration	Params	Lat. (ms)	Peak val.	Gap
NB2	baseline	2.19M	0.380	91.11	8.87
NB2	depth 2blocks	4.22M	0.302	86.74	13.21
NB2	depth 4blocks	1.44M	0.471	92.37	7.32
NB2	depth 5blocks	1.24M	0.635	93.04	6.17
NB2	fc 1layers	0.18M	0.338	90.74	9.21
NB2	fc 3layers	2.22M	0.438	91.11	8.79
NB2	kernel 5x5	2.36M	0.535	89.16	10.81
NB2	kernel 7x7	2.61M	0.771	79.95	15.87
NB2	padding valid	1.28M	0.391	89.21	10.51
NB2	downsample stride	2.19M	0.321	84.37	15.43
NB2	batchnorm	2.19M	0.523	91.90	7.68
NB2	combined	1.24M	0.735	94.52	4.89
NB3	SGD lr0.01	1.24M	0.694	93.28	5.49
NB3	Adam lr0.001	1.24M	0.694	94.15	2.47
NB3	AdamW lr0.001	1.24M	0.694	94.00	4.15
NB3	Adam lr0.001 StepLR	1.24M	0.694	95.85	3.31
NB3	Adam lr0.001 Cosine	1.24M	0.694	95.85	3.92
NB3	Adam lr0.001 OneCycle	1.24M	0.694	95.53	4.40
NB4	none	1.24M	0.694	95.85	3.90
NB4	flips	1.24M	0.694	96.30	1.85
NB4	flips+crop	1.24M	0.694	96.44	1.43
NB4	flips+crop+affine	1.24M	0.694	96.10	0.96
NB4	flips+crop+affine+jitter	1.24M	0.694	95.65	0.99
NB4	flips+crop+mixup	1.24M	0.694	96.52	1.21
NB4	flips+crop+cutmix	1.24M	0.694	95.19	0.91

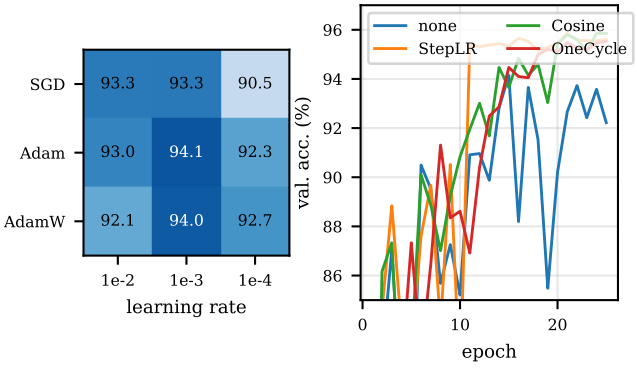


Fig. 4. NB3. Left: peak validation accuracy (%) for each optimizer and initial learning rate. Right: validation accuracy of Adam (lr 1e-3) with each scheduler.

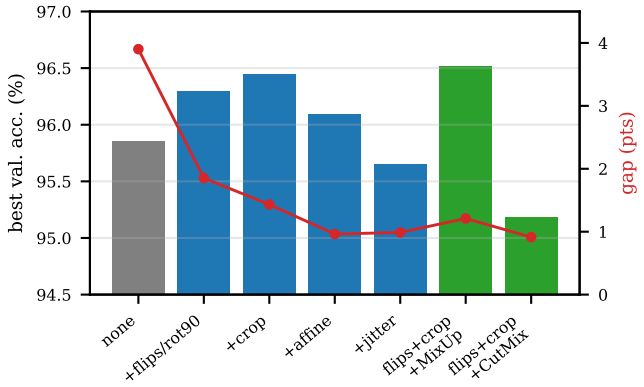


Fig. 6. NB4 augmentation ablation (cumulative from left to right; MixUp/CutMix are added to flips+crop). Bars: peak validation accuracy; red: gap measured on clean training images.

C. Data augmentation (NB4)

Satellite tiles have no canonical orientation, so unlike CIFAR-10 **vertical flips and 90° rotations preserve the label** (Fig. 5). We add augmentations cumulatively (Fig. 6): flips + rotations by multiples of 90°, `RandomCrop(64, padding=4, reflect)`, `RandomAffine(±15°, scale 0.9-1.1)` and a weak `ColorJitter` (hue ±0.02, because color carries the class: vegetation is green, water blue). MixUp [9] and CutMix [10] are implemented by hand in the training loop, with the mixed loss $\lambda\ \mathrm{CE}(\hat{y}, y_a) + (1-\lambda)\ \mathrm{CE}(\hat{y}, y_b)$. Because the training accuracy measured on augmented images underestimates overfitting, the gap is computed on a fixed subset of 4,050 **un-augmented** training images.

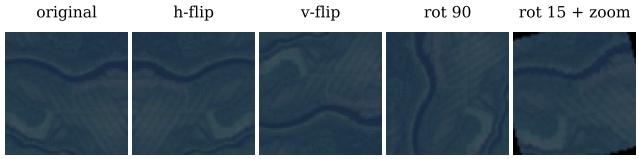


Fig. 5. Label-preserving transformations of a River tile. Flips and 90° rotations are exact; a 15° rotation creates black corners.

Flips and rotations alone halve the gap (3.9 → 1.9 points) and the gap keeps decreasing down to ≈1 point. Accuracy, however, peaks with flips+crop (96.44%): the ±15° affine rotation creates black corners never seen at test time, and color jitter slightly destroys useful information. MixUp on top of flips+crop gives the best model (96.52% validation, **97.04% test**). CutMix had the smallest gap but its validation accuracy was still increasing at epoch 25: such strong regularization needs longer training.

D. Task 2 comparison

Table I summarizes all Task 2 configurations. Latency is measured on the T4 GPU for one image; since all NB3/NB4 runs share the same network, they have the same size and latency.

V. TASK 3 — TRANSFER LEARNING (NB5)

We fine-tune an ImageNet-pretrained ResNet-18 [11] (11.2M parameters), following the structure of the course script: the parameters are frozen with `requires_grad = False` and the last layer is replaced by `Dropout(0.5)` → `Linear(512, 10)`. Because ResNet-18 divides the resolution by 32, the tiles are upsampled to 128×128 (4×4 final maps) and normalized with the ImageNet statistics. We compare (i) a **frozen** backbone (5,130 trainable parameters: a linear classifier on ImageNet features), (ii) **partial** fine-

TABLE III

REPRODUCTION OF NASWOT ON NAS-BENCH-201. TOP: KENDALL τ BETWEEN THE SCORE AND THE VALIDATION ACCURACY (1000 NETWORKS). BOTTOM: TEST ACCURACY (%) OF THE NETWORK SELECTED BY THE SEARCH (MEAN ± STD OVER 500 RUNS), OURS VS. TABLE 2 OF [2]. SEARCH TIME IS FOR SCORING N NETWORKS ON A T4 GPU.

Kendall τ	**CIFAR-10**		**CIFAR-100**		**ImageNet16-120**	
	ours	paper	ours	paper	ours	paper
NASWOT score	0.602	0.574	0.640	0.611	0.600	0.549
Search (test acc.)	ours	paper	ours	paper	ours	paper
NASWOT (N=10)	92.39±1.20	92.44±1.13	68.63±2.31	68.62±2.04	41.29±3.94	41.31±4.11
NASWOT (N=100)	92.83±0.94	92.81±0.99	69.52±1.67	69.48±1.70	43.18±2.95	43.10±3.16
NASWOT (N=1000)	92.91±1.03	92.96±0.81	69.84±1.42	69.98±1.22	43.99±2.70	44.44±2.10
Random	87.04±12.94	86.61±13.46	61.36±12.17	60.83±12.58	33.54±9.52	33.13±9.66
Optimal (N=100)	93.86±0.22	93.84±0.23	71.59±0.74	71.56±0.78	45.64±0.60	45.67±0.64

tuning of `layer4` and (iii) **full** fine-tuning, with a learning rate of 1e-4 for pretrained layers and 1e-3 for the new head, to avoid destroying the pretrained features. The reference is our best from-scratch CNN. To measure **data efficiency** every method is trained on nested stratified subsets of 5/10/25/100% of the training set, with more epochs for smaller subsets (60/50/30/15) and the same augmentation (flips+crop) and cosine schedule for all.

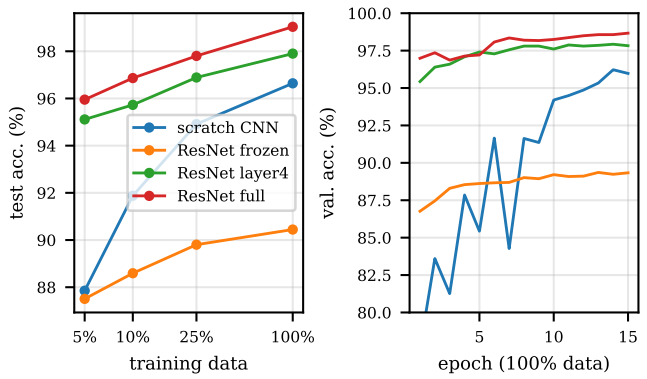


Fig. 7. NB5. Left: test accuracy vs. amount of training data. Right: validation accuracy per epoch with 100% of the data.

TABLE II

NB5 TEST ACCURACY (%) FOR EACH FRACTION OF THE TRAINING DATA, AND EPOCHS TO REACH 90% VALIDATION ACCURACY WITH 100% OF THE DATA.

Method	**Train. params**	**5%**	**10%**	**25%**	**100%**	**Ep.→90%**
Scratch CNN	1.24M	87.85	91.88	94.91	96.64	6
ResNet frozen	5.1k	87.51	88.59	89.80	90.44	never
ResNet layer4	8.40M	95.11	95.73	96.89	97.90	1
ResNet full	11.18M	95.95	96.86	97.80	99.04	1

Full fine-tuning is the best strategy at every data size (Table II): **99.04% test accuracy (macro-F1 98.99)** with all the data, and 95.95% with only 5% (945 images) — about the accuracy of our from-scratch CNN trained on 20 times more data. Fine-tuned models also converge much faster: they exceed 90% validation accuracy after the first epoch, against 6 epochs from scratch (Fig. 7, right). The frozen backbone, in contrast, saturates around 90%: the last ImageNet layers encode object parts (eyes, wheels) that a linear classifier cannot map to subtle land-use textures, and it is overtaken by the from-scratch CNN as soon as 10% of the data is available. Unfreezing only `layer4` recovers most of the gain, which confirms that the **last** layers are the dataset-specific ones. The price of transfer learning is size and speed: ResNet-18 has 9× more parameters and is 3.4× slower than our CNN (2.5 vs. 0.74 ms per image). A side remark: the training accuracies of NB5 are measured on augmented images with dropout active, which makes the frozen model's apparent gap negative; NB4′s clean-image measurement should be preferred.

VI. ZERO-COST NEURAL ARCHITECTURE SEARCH

A. Method

NASWOT [2] scores an **untrained** network on one mini-batch of B images. Each image i gets a binary code c_i indicating which of the N_A ReLUs are active; images with the same code lie in the same linear region of the network and cannot be separated by it. With the kernel $K_H[i,j] = N_A - d_H(c_i, c_j)$ (d_H: Hamming distance), the score is $s = \log|K_H|$: a high score means the network already separates the inputs well at initialization. We implement it with forward hooks on every `nn.ReLU`, accumulating $xx^T + (1-x)(1-x)^T$ in double precision. For comparison we also implement SynFlow [12] (data-free), grad-norm and SNIP [13], and the number of parameters as a baseline.

B. Reproduction on NAS-Bench-201 (NB6.0–NB6.1)

NAS-Bench-201 [3] contains 15,625 cell-based networks whose accuracies after 200 epochs on CIFAR-10, CIFAR-100 and ImageNet16-120 are published. We re-implemented its networks from the paper's description; for 300 random architectures our parameter counts match the official ones exactly. Following the paper, we score 1000 random networks on a mini-batch of 128 images of each dataset and compute Kendall's τ with the validation accuracy (ImageNet16 is not available on Kaggle, so for this dataset we score with CIFAR-100 images downsampled to 16×16). For the search, the paper samples N networks, keeps the one with the highest score, and averages over 500 runs; we score all 15,625 networks once and simulate the 500 runs from these scores.

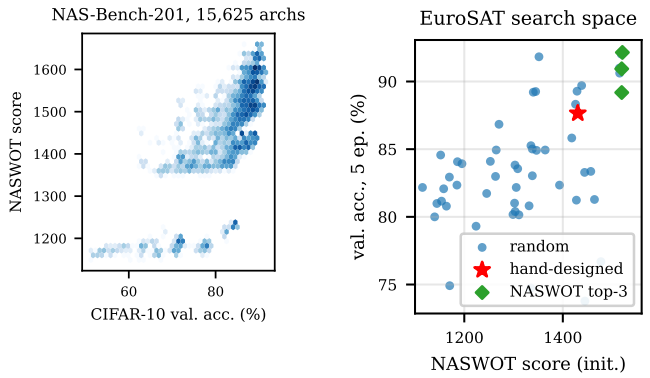


Fig. 8. NASWOT score at initialization vs. trained accuracy. Left: all NAS-Bench-201 networks. Right: our EuroSAT study (5-epoch training).

Table III shows that the reproduction is successful. Our correlations are slightly **higher** than the published ones (0.60/0.64/0.60 vs. 0.57/0.61/0.55; differences of the sampled networks and of the mini-batches), and the search results agree with the paper to within ≈0.1 point for $N = 10$ and $N = 100$ on all three datasets, far inside the standard deviations. Our "Random" and "Optimal" references also match, which validates the benchmark extraction. With the same 1000 networks, the comparison proxies keep the ranking reported by [14] (Spearman ρ on CIFAR-10 test: SynFlow 0.80 vs. 0.74 published, grad-norm 0.65 vs. 0.58, SNIP 0.65 vs. 0.58), and NASWOT reaches ρ = 0.81. Finally, the ranking of 100 networks computed with five different mini-batches is almost identical (mean τ = 0.991), which justifies scoring each network only once.

C. Zero-cost NAS on EuroSAT (NB6.2)

Our search space turns the Task 2 knobs into choices: 2–5 blocks, a kernel size of 3/5/7 **per block**, a width of 16/32/64 channels (doubled at each block), BatchNorm on/off, max-pooling or stride-2 downsampling, and an optional residual connection (1×1 projection) — 8,640 networks, including our hand-designed one. Since no ground truth exists, we trained 50 random networks + the hand-designed one for 5 epochs (Adam 1e-3, no augmentation) and correlated their validation accuracy with the proxies (Fig. 8, right):

TABLE IV
EuroSAT: rank correlation between each proxy and the accuracy after a 5-epoch training (51 networks).

	NASWOT	SynFlow	SNIP	grad-norm	#params
Spearman ρ	0.43	0.09	0.51	0.56	−0.13
Kendall τ	0.32	0.07	0.38	0.40	−0.08

NASWOT, SNIP and grad-norm are moderately correlated with accuracy ($\rho \approx 0.43$–0.56), while the number of parameters is not ($\rho = -0.13$): the proxies measure more than size. SynFlow fails here ($\rho = 0.09$); unlike in NAS-Bench-201 (global pooling), our networks end with large flatten+FC layers that dominate its product of absolute weights. Correlations are lower than on NAS-Bench-201, but our ground truth (5 epochs) is itself noisy.

Search. We scored 1000 random networks with all proxies in 153 s, trained the top-3 by NASWOT and by SynFlow for 5 epochs, and compared with 3 random networks and the hand-designed CNN under the same budget:

TABLE V
Search on EuroSAT: validation accuracy (%) after 5 epochs of the selected networks.

Selection	Mean	Best
NASWOT top-3	90.76	92.15
SynFlow top-3	87.96	91.63
Random (3)	83.32	84.77
Hand-designed	87.65	87.65

All three NASWOT picks beat both the random networks and our hand-designed CNN. The winner (5 blocks with kernels 5-3-3-3-3, width 64, BatchNorm, max-pooling, residual connections) was then trained with the full NB4 recipe, alongside the hand-designed network in the same run: **97.43% test accuracy vs. 96.72%**, with 1.99M vs. 1.24M parameters and 1.31 vs. 0.76 ms latency. The NAS needed 17 GPU-minutes (scoring + 6 short trainings) against 203 minutes for the 31 trainings of NB2–NB4 — about 12 times less. The search rediscovered the main lessons of Task 2 (depth 5, small kernels, max-pooling, BatchNorm) and added the two changes we had not tried: more width and residual connections.

VII. Implementation and reproducibility

The project is organized as nine notebooks (Table VI), each self-contained so that it can be executed live: helper functions (`train_one_epoch`, `evaluate`, `measure_latency`) are defined inside each notebook, training loops are written explicitly (`zero_grad`, forward, `backward`, `step`), and every trained configuration is saved with `torch.save(net.state_dict())`. Notebooks are pushed to Kaggle with the command-line API, executed on a T4 GPU, and their outputs (executed notebook, checkpoints, JSON histories) are downloaded; later notebooks read the results of earlier ones (e.g. NB3 starts from the architecture chosen in NB2), which makes the whole study reproducible end-to-end from the fixed split and seed. All figures and tables of this report are generated from these JSON files.

TABLE VI
Notebooks of the project and number of trained configurations (all saved as checkpoints, except the 5-epoch NAS evaluations).

NB	Content	Runs
0	split, normalization statistics	–
1	baseline CNN (Task 1)	1
2	architecture variations (Task 2a)	12
3	optimizers × lr, schedulers (Task 2b)	12
4	augmentation, MixUp/CutMix (Task 2c)	7
5	transfer learning × data fraction (Task 3)	16
6.0/6.1	NAS-Bench-201 extraction + NASWOT reproduction	–
6.2	zero-cost NAS on EuroSAT	60 + 2
7	evaluation of all checkpoints, live demo	–

VIII. Final comparison and discussion

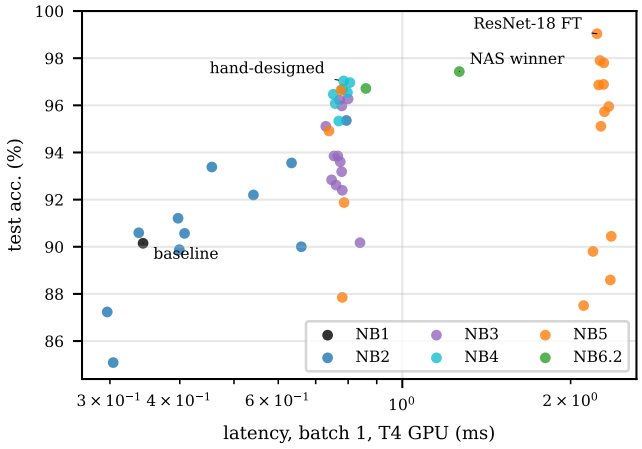


Fig. 9. NB7: test accuracy vs. single-image latency of all 50 checkpoints, re-evaluated on the same test set.

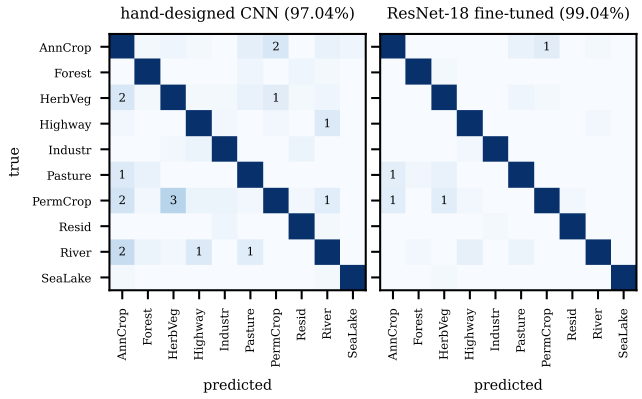


Fig. 10. Normalized test confusion matrices of the hand-designed CNN (left) and the fine-tuned ResNet-18 (right).

TABLE VII
The three final models on the test set (4,050 images), re-evaluated in NB7. Latency: one image on a T4 GPU.

Model	Acc.	F1	Params	Lat.	Errors
Baseline (NB1)	90.15	89.75	2.19M	0.34 ms	399
Hand-designed (NB4)	97.04	96.97	1.24M	0.79 ms	120
NAS winner (NB6.2)	97.43	97.36	1.99M	1.26 ms	104
ResNet-18 frozen	90.44	89.98	11.18M	2.36 ms	387
ResNet-18 fine-tuned	99.04	98.99	11.18M	2.23 ms	39

NB7 reloads all 50 checkpoints and re-evaluates them on the test set (Fig. 9, Table VII); every number matches the one obtained in its own notebook. The final models offer three trade-offs: the hand-designed CNN (97.04%, 1.24M parameters, 0.79 ms) is the fastest, the NAS winner removes 13% of its errors for a 1.6× slower network, and the fine-tuned ResNet-18 is by far the most accurate (99.04%, 39 errors instead of 120) but also the largest and slowest. The confusion matrices (Fig. 10) show that, once Highway/River has been largely fixed by depth and augmentation, the remaining errors of the from-scratch models concentrate on the agricultural classes (PermanentCrop→HerbaceousVegetation, Pasture/River→AnnualCrop), whose 64×64 textures are genuinely ambiguous; ImageNet features reduce these confusions by a factor of 2–3. The two from-scratch models share most of their errors (72 in common), whereas ResNet-18 fails on different images (only 15–17 in common with each), which suggests that an ensemble of a scratch CNN and the ResNet could still help. 14 test images are misclassified by all three models; inspecting them in NB7, most are patchworks of fields confused between AnnualCrop, Pasture and PermanentCrop, plus a river running along a road and an industrial area crossed by a large road: tiles mixing several land uses, for which a single label is debatable.

Limitations. Each configuration was trained with a single seed, while repeated runs differ by up to ≈1 point, so small differences (e.g. between the top augmentation settings, or StepLR vs. cosine) are not significant; the large effects (depth, kernel size, scheduler, augmentation on the gap, transfer learning, NAS vs. random) are well beyond this noise. Our EuroSAT NAS ground truth uses a short training, and the ImageNet16 reproduction uses substitute images.

IX. Conclusion

Careful hand-tuning took a simple CNN from 90.15% to 97.04% test accuracy: depth and small kernels gave the largest architectural gains, Adam with a decaying learning rate the best optimization, and orientation-invariant augmentation removed most of the overfitting. Transfer learning from ImageNet remains the most accurate and by far the most data-efficient option (95.95% with 5% of the data), at a 9× size cost. Finally, after reproducing the published results of training-free NAS on NAS-Bench-201, we showed that it transfers to a new problem: in minutes, it found a CNN better than the one we designed by hand in hours.

References

[1] P. Helber, B. Bischke, A. Dengel, and D. Borth, "EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 12, no. 7, pp. 2217–2226, 2019.

[2] J. Mellor, J. Turner, A. Storkey, and E. J. Crowley, "Neural Architecture Search without Training," in *International Conference on Machine Learning (ICML)*, 2021, pp. 7588–7598.

[3] X. Dong and Y. Yang, "NAS-Bench-201: Extending the Scope of Reproducible Neural Architecture Search," in *International Conference on Learning Representations (ICLR)*, 2020.

[4] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.

[5] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[6] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

[7] I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," in *International Conference on Learning Representations (ICLR)*, 2017.

[8] L. N. Smith and N. Topin, "Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates," in *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, 2019.

[9] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond Empirical Risk Minimization," in *International Conference on Learning Representations (ICLR)*, 2018.

[10] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 6023–6032.

[11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[12] H. Tanaka, D. Kunin, D. L. K. Yamins, and S. Ganguli, "Pruning Neural Networks without any Data by Iteratively Conserving Synaptic Flow," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[13] N. Lee, T. Ajanthan, and P. H. S. Torr, "SNIP: Single-shot Network Pruning based on Connection Sensitivity," in *International Conference on Learning Representations (ICLR)*, 2019.

[14] M. S. Abdelfattah, A. Mehrotra, Ł. Dudziak, and N. D. Lane, "Zero-Cost Proxies for Lightweight NAS," in *International Conference on Learning Representations (ICLR)*, 2021.